DS605 · FUNDAMENTALS OF MACHINE LEARNING

Lecture Notes

Evaluation — I

Schemes for Data Split and Handling Bias–Variance

Lecture 09–10
20 August 2026

🛒 **Running example:** *Shopkart, the purchase-prediction model introduced in Lectures 07–08*

Contents

1. Where This Lecture Fits 3
2. The Learner's Objective, Revisited 3
3. Measuring Error: Sample Error vs. True Error 3
4. Why We Need a Held-Out Test Set 4
5. The Holdout Method 4
6. Facts About the Holdout Method 4
7. K-fold Cross-Validation 5
8. Nested K-fold Cross-Validation 5
9. Diagnosing Model Behavior 6
10. Evaluation Metrics: The Big Picture 7
11. Measures for Regression Problems 7
12. Measures for Classification Problems 7
13. Precision / Recall Trade-off 9
14. Choosing a Threshold 9
15. The ROC Curve 9
16. AUC: Area Under the ROC Curve 9
17. Key Takeaways 10
18. Looking Ahead: Bridge to Evaluation II 10

1. Where This Lecture Fits

Lectures 07–08 covered representation: hypothesis spaces, inductive bias, and the bias–variance trade-off. Having built a hypothesis, the natural next question is how well it will actually work once it leaves the classroom. That question splits into two lectures.

Lectures	Theme
07–08	Representation — hypothesis spaces, inductive bias, bias–variance
09–10 (this lecture)	Evaluation I — how do we split data and measure generalization?
11–12	Evaluation II — how do we weigh different mistakes? (Loss Functions)

Shopkart remains the linked example. Every split, every metric introduced below is grounded in the purchase-prediction model from Lectures 07–08.

2. The Learner's Objective, Revisited

A learner searches a hypothesis space for a final hypothesis. For that hypothesis to be worth deploying, it needs two properties:

Consistent

The hypothesis fits the training data. This is necessary — but not sufficient on its own.

Generalizes well

The hypothesis performs well on data it has never seen. This is what actually matters in practice: nobody deploys a model to re-predict the data it was trained on.

Experimental evaluation is exactly the set of tools we use to measure generalizability — and building that toolkit is the whole job of this lecture.

3. Measuring Error: Sample Error vs. True Error

Sample error of hypothesis *h* on a fixed, finite sample *S* is the fraction of that sample *h* gets wrong:

$$error_S(h) = \frac{1}{n} \sum_{x \in S} \delta(h(x), f(x))$$

where δ is the indicator function — it returns 1 when its two arguments disagree and 0 when they match (formally defined in Section 12, alongside the Misclassification Rate).

True error is the probability that *h* misclassifies a single example drawn at random from the full underlying distribution *D*:

$$error_D(h) = P_{x \in D}[h(x) \neq f(x)]$$

True error is the number we actually care about — but we can never compute it directly, because we don't have access to the entire distribution D. Every evaluation scheme in this lecture is, at bottom, a strategy for estimating error_D(h) using only a finite sample.

4. Why We Need a Held-Out Test Set

Training error

Also called resubstitution error or in-sample error, it is measured on the very data the model was fit on. The model has already "seen the answers" — so it flatters itself. Training error is an optimistic (systematically too low) estimate of true error.

Test error

Also called out-of-sample error or extra-sample error, it is measured on data that played no part whatsoever in building the model. This is what gives us an honest estimate of generalization, which is precisely why a held-out test set is non-negotiable.

5. The Holdout Method

The simplest evaluation scheme: partition the available data once into a training split (e.g., 80%) and a test split (the remaining 20%). Train the estimator only on the training split. Evaluate its predictions only on the test split — and only once.

CONCEPT CHECK · WHEN SHOULD WE STANDARDIZE?

The question: when standardizing features (subtracting the mean, dividing by the standard deviation), does it matter whether you standardize before or after splitting into train/test?

Answer: fit the mean and standard deviation on the training set only, then apply that same transform to the test set. Any ordering that computes statistics on the full dataset (train + test combined) before splitting lets information from the test set leak into the training set — the model implicitly "sees" test-set information before it should.

Why it matters: This mirrors deployment. In production, the model will never have access to future data in advance either.

Rule of thumb: *any statistic used to transform the data must be learned after the split, from the training data alone.*

6. Facts About the Holdout Method

Results vary across runs

A single split might be “lucky” or “unlucky” — the train or test half may not be representative of the whole distribution.

Pessimistic on small data

Training on as little as 50% of the data can produce a noticeably worse model than one trained on the full dataset, simply because it has seen less data.

7. K-fold Cross-Validation

Shuffle the dataset and partition it into k disjoint, equal-sized folds — typically k = 10. Run k rounds: in each round, one fold serves as the held-out test fold and the remaining k − 1 folds are used for training. Rotate which fold plays the test role every round, so that every example is tested exactly once.

Pseudocode

```
Shuffle dataset D, partition into k disjoint, equal-sized subsets D₁, …, D_k

for i = 1 to k:
    train on   D \ Dᵢ
    predict for  Dᵢ
    measure error  (e.g., MAE)

report the mean of the k errors
```

The final reported number is a mean over k independent estimates of generalization error — already more trustworthy than a single holdout split.

Facts About K-fold Cross-Validation

Fold size is constrained

Statisticians often want folds of at least ~30 examples — this caps the maximum value of k for a given dataset.

Costly

The learning algorithm must be trained k separate times, which multiplies the compute budget by k.

Still some variance

A particular shuffle can still be “lucky” or “unlucky” — just less so than a single holdout split.

8. Nested K-fold Cross-Validation

When we need to tune a hyperparameter — a parameter of the model class itself, not of one fitted model — a single train/test split is no longer enough. We split the data into three disjoint sets instead:

Set	Role
Training set	Train candidate models
Validation / dev set	Evaluate candidates, pick the best
Test set	Final, unbiased evaluation

The workflow proceeds in three stages:

- **Train + Select —** find the best model on the dev set (this is the *inner* cross-validation loop).
- **Merge, Train, Test —** retrain on training + validation data, then evaluate once on the held-out test set (the *outer* loop).
- **Merge, Train, Deploy —** once satisfied, retrain on *all* available data for the model that actually ships.

K-fold CV can be applied to the validation split (inner) and the test split (outer) simultaneously — hence "nested."

9. Diagnosing Model Behavior

Reading the Error Curves

Comparing training, validation, and test error side by side tells you which regime you're in:

Situation	Training error	Validation error	Test error
Underfitting	high	high	high
Good model	low	low	low
Overfitting	low	high	high
IID violation	low	low	high

Underfitting vs. Overfitting: What to Do

	Underfitting	Overfitting
Model complexity	Need a more complex model	Need a simpler model
Regularization	Need less regularization	Need more regularization

	Underfitting	Overfitting
Features	Need more features	Remove extra features
More data	Doesn't help	Usually helps

Keep these two columns as a checklist: before touching the model, first identify which side of the curve you're on.

10. Evaluation Metrics: The Big Picture

Once we trust the number (Sections 5–9 were all about that), the next question is whether it's even the right number to be looking at.

Metric family	What it captures
Error	How wrong, on average
Accuracy	Fraction of correct predictions
Precision / Recall	Correctness by class, under imbalance

Choosing a split scheme (Sections 5–9) answers **"Is this number trustworthy?"** Choosing a metric (Sections 11–16) answers **"Is this the right number to be looking at?"**

11. Measures for Regression Problems

Mean Absolute Error weights every error linearly:

$$MAE = \frac{1}{n} \sum_{x \in S} |h(x) - y|$$

Mean Squared Error weights larger errors much more heavily, because the error term is squared:

$$MSE = \frac{1}{n} \sum_{x \in S} (h(x) - y)^2$$

Root Mean Squared Error brings the units back in line with the original target by undoing the square:

$$RMSE = \sqrt{MSE}$$

12. Measures for Classification Problems

Misclassification Rate (a.k.a. Error Rate)

$$MR = \frac{1}{n} \sum_{x \in S} \delta(h(x), y)$$

where δ is the indicator function:

$$\delta(h(x), y) = 1 \text{ if } h(x) \neq y, \text{ and } \delta(h(x), y) = 0 \text{ otherwise.}$$

Simple and intuitive — but it treats every mistake as equally costly, which is rarely true in practice. That gap is the seed of Lecture 11–12.

The Confusion Matrix

For binary classification, every prediction falls into one of four buckets:

Predicted \ Actual	Actual: Positive	Actual: Negative
Predicted: Positive	TP	FP
Predicted: Negative	FN	TN

From these four counts, we can derive every classification metric in this section:

$$accuracy = \frac{TP + TN}{P + N}$$

$$error\ rate = 1 - accuracy = \frac{FP + FN}{P + N}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$Specificity = \frac{TN}{FP + TN}$$

Specificity is the recall of the negative class — the fraction of true negatives correctly identified.

F-measure: Balancing Precision and Recall

The F measure is the weighted harmonic mean of precision and recall:

$$F = \frac{1}{\alpha \cdot \frac{1}{Precision} + (1-\alpha) \cdot \frac{1}{Recall}} = \frac{(\beta^2+1) \cdot Precision \cdot Recall}{\beta^2 \cdot Precision + Recall}$$

CONCEPT CHECK · WHICH METRIC MATTERS?

The question: what metric would you use for each of the following classifiers?

- **Kid-safe video classifier —** a missed unsafe video (a false negative) is far worse than mistakenly flagging a safe one. → **optimize Recall.**
- **Shoplifter-detection classifier —** a false accusation (a false positive) has serious real-world consequences for an innocent shopper. → **optimize Precision.**

There is no universal "best" metric — the cost of each error type decides which one to optimize.

13. Precision / Recall Trade-off

Concrete example: a classifier scores images of handwritten digits by how confident it is that each one is a "5." Sort the images by score, and everything above a chosen threshold is predicted positive. Moving the threshold to the right — requiring higher confidence — leaves fewer, more confident positives: precision rises, but recall falls.

Threshold	Precision	Recall
Low	6/8 = 75%	6/6 = 100%
Medium	4/5 = 80%	4/6 = 67%
High	3/3 = 100%	3/6 = 50%

14. Choosing a Threshold

A high-precision classifier isn't useful if its recall is too low — precision alone is never the full story. Plotting precision against recall across every possible threshold (the precision–recall curve) shows the entire achievable trade-off at a glance, rather than a single operating point.

15. The ROC Curve

The receiver operating characteristic (ROC) curve plots the true positive rate (TPR = Recall) against the false positive rate (FPR = 1 − Specificity) as the decision threshold is varied over its full range.

Random classifier

Traces the diagonal — TPR grows exactly as fast as FPR, meaning the classifier carries no information.

Good classifier

Bows toward the top-left corner — high TPR at low FPR, meaning the classifier is separating the classes well.

16. AUC: Area Under the ROC Curve

AUC compresses the whole ROC curve into a single number: the probability that the classifier ranks a randomly chosen positive example above a randomly chosen negative one.

Perfect classifier

Hugs the top-left corner, so the area under its curve approaches 1.

Random classifier

Traces the diagonal, so the area under its curve is 0.5.

17. Key Takeaways

- Holdout is simple but variable; k-fold CV trades compute for a more reliable estimate; nested CV is needed the moment you tune hyperparameters.
- Compare training / validation / test error together to diagnose underfitting, overfitting, or a broken IID assumption.
- Regression: MAE, MSE, RMSE. Classification: accuracy, precision, recall, F-measure — pick based on the real cost of each error.
- PR and ROC curves show the full threshold trade-off, not just one operating point.

18. Looking Ahead: Bridge to Evaluation II

COMING UP · LECTURE 11–12

Evaluation — II: Loss Functions

Misclassification rate and plain accuracy treat every mistake as equally costly — Section 12 already flagged why that's rarely true. Evaluation II asks the deeper question: how costly is each kind of mistake, and how do we build that cost directly into what the model optimizes? The next lecture builds loss functions from first principles — MSE, MAE, and Huber loss for regression; entropy, cross-entropy, and KL divergence for classification — continuing to ground every idea in the Shopkart running example.